UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SECP3623-02 DATABASE PROGRAMMING

SEMESTER 01 2025/2026

PROJECT 2

GROUP 3

LECTURER: DR. SHAMINI A/P RAJA KUMARAN

NAME	MATRIC NO
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201
AIN NURNABILA BINTI MOHD AZHAR	A23CS0207
SYARIFAH DANIA BINTI SYED ABU BAKAR	A23CS0183
NUR FIRZANA BINTI BADRUS HISHAM	A23CS0156

VIDEO LINK:

https://drive.google.com/drive/folders/1M6td-pRaNE9RN37vGEef9M0pIlR-X5W7?usp=sharing

Table of Contents

1.0 Introduction

Briefly describe the reason behind the chosen domain, stated the project objectives and assigned the listed team member to each task equally.

1.1 Brief description of the chosen domain

The delivery system is chosen as our domain because it is a widely used real-world application that involves the management of users, orders, restaurants, and delivery partners. Modern delivery systems are commonly used in food delivery and e-commerce platforms, where large volumes of data are generated and updated in real time. This makes the domain suitable for NoSQL databases such as MongoDB, which are designed to handle flexible data structures and high scalability.

1.2 Project objectives

The objectives of this project are as follows:

1. To design a NoSQL database backend using MongoDB based on a selected real-world domain, demonstrating the application of the Document Data Model.
2. To create and manage MongoDB collections that utilize embedded documents, arrays, references, and appropriate data types.
3. To perform CRUD operations (Create, Read, Update, Delete) and demonstrate data manipulation using MongoDB query and update operators.
4. To implement indexing and aggregation pipelines for improved performance and analytics in the database.
5. To analyse security considerations and design challenges associated with NoSQL databases and document-oriented data modeling.

1.3 Team members & roles

Table 1.3 shows the team composition and specific tasks for each member:

Team member	Assigned roles	Description
Yasmin Batrisyia	NoSQL Data Modelling	Design collection and provide sample documents

Nur Firzana	Basic MongoDB Operations (CRUD)	Implement CRUD operations related to domain such as insert, query, update and delete
Syarifah Dania	Advanced MongoDB Operations - Indexing - Aggregation Pipelines - Sorting	Create indexes, deliver aggregation outputs, and sort records in descending and ascending order.
Ain Nurnabila	Advanced MongoDB Operations - Query Performance Discussion - Security & Challenges	Explain how indexes influence query speed, discuss about security and challenges in NoSQL

2.0 NoSQL Data Model

The NoSQL data model is implemented using MongoDB, a document-based database that stores data in collections and JSON-like documents.

2.1 List and explanation of collections

Databases are used to store related information. SQL stores data in tables, while MongoDB stores information in collections. A database can have one or many collections, and each collection must have a unique name. Collections store information about objects or entities such as users, restaurants and so on. Furthermore, a collection can have any number of documents, each collection has a unique id and its own data. Figure 2.1 shows a list of collections that is created.

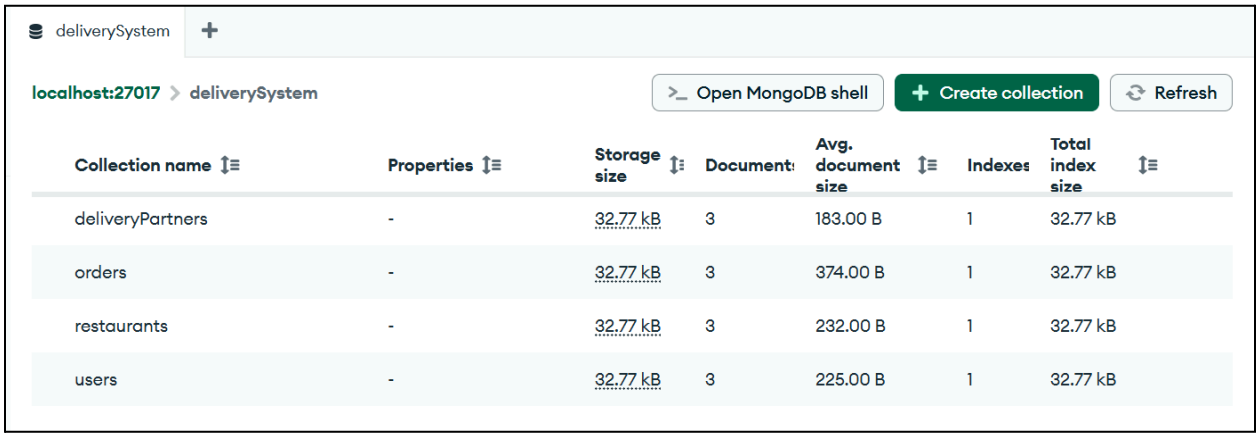


Figure 2.1 List of collections

2.2 Example JSON documents

Documents are similar to rows or records in SQL tables. Figures below shows an examples of JSON documents created in each collection.

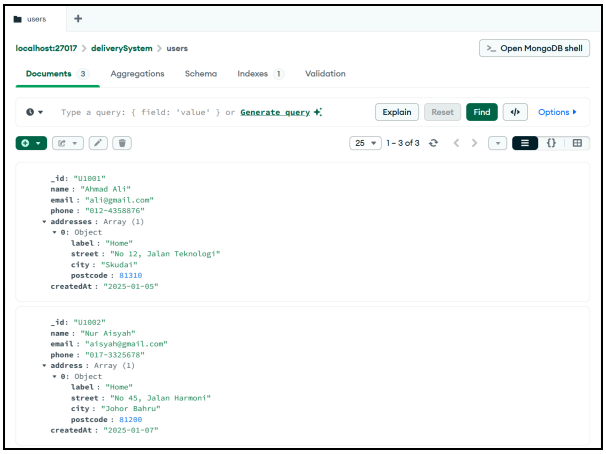


Figure 2.2.1 documents in users

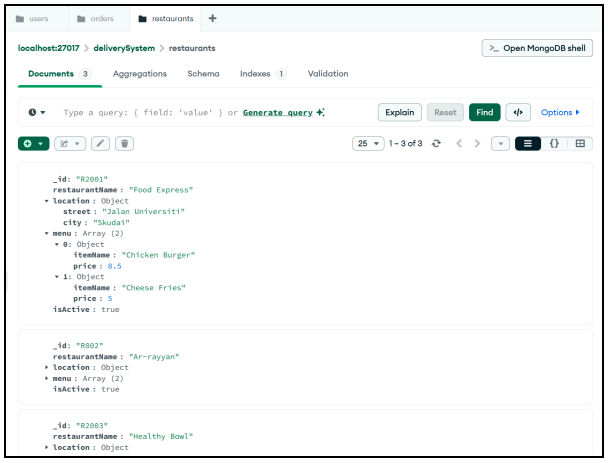


Figure 2.2.2 documents in restaurants

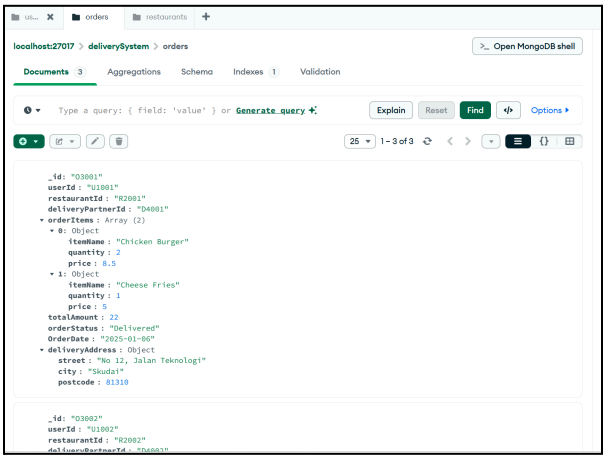


Figure 2.2.3 documents in orders

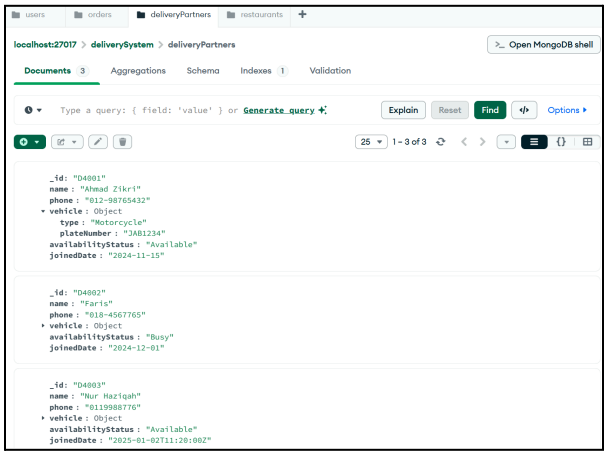


Figure 2.2.4 documents in deliveryPartners

2.3 Explanation of embedding vs referencing

Embedding stores related data within a single document by placing documents inside other documents. This approach allows related information to be retrieved together in a single query, which improves read performance and reduces the need for JOIN operations. Embedding is suitable when the embedded data is frequently accessed together with the main document and does not change often. Figure 2.3.1 shows embedded documents in user, restaurants, orders and deliveryPartners collection.

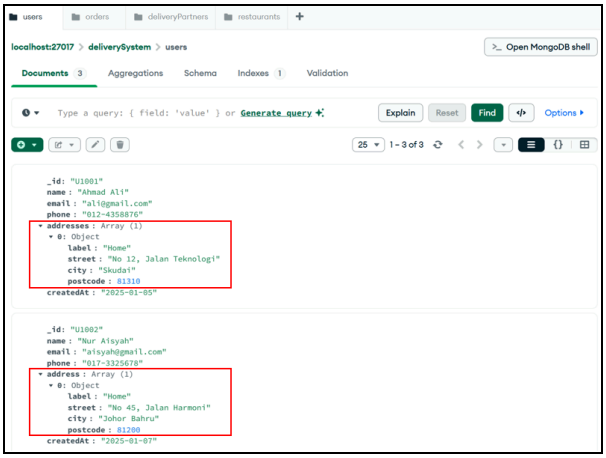


Figure 2.3.1.1 Embedded documents in users

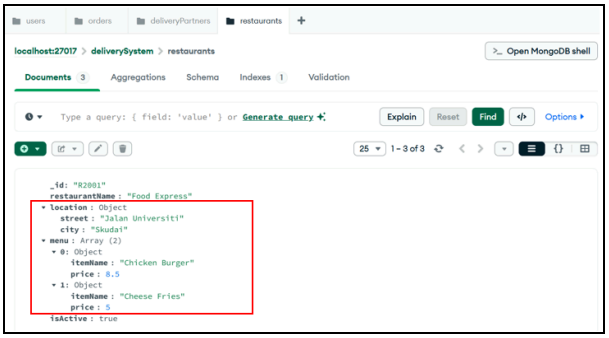


Figure 2.3.1.2 Embedded documents in restaurants

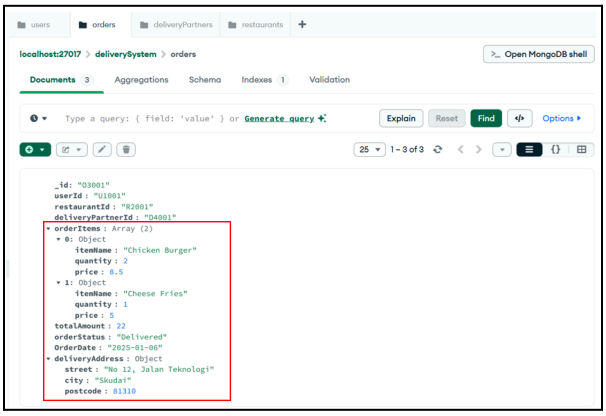


Figure 2.3.1.3 Embedded documents in orders

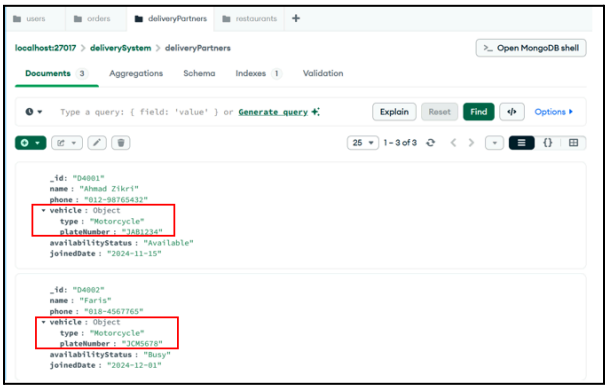


Figure 2.3.1.4 Embedded documents in deliveryPartners

Referencing, known as the normalized data model, which stores related data in separate collections. Documents are linked using ObjectID references, typically through the _id field. This approach avoids data duplication and improves data consistency, making it suitable for large or frequently changing data. Referencing is similar to normalization in relational databases and is useful when related data is shared across multiple documents. Figure 2.3.2 shows references documents.

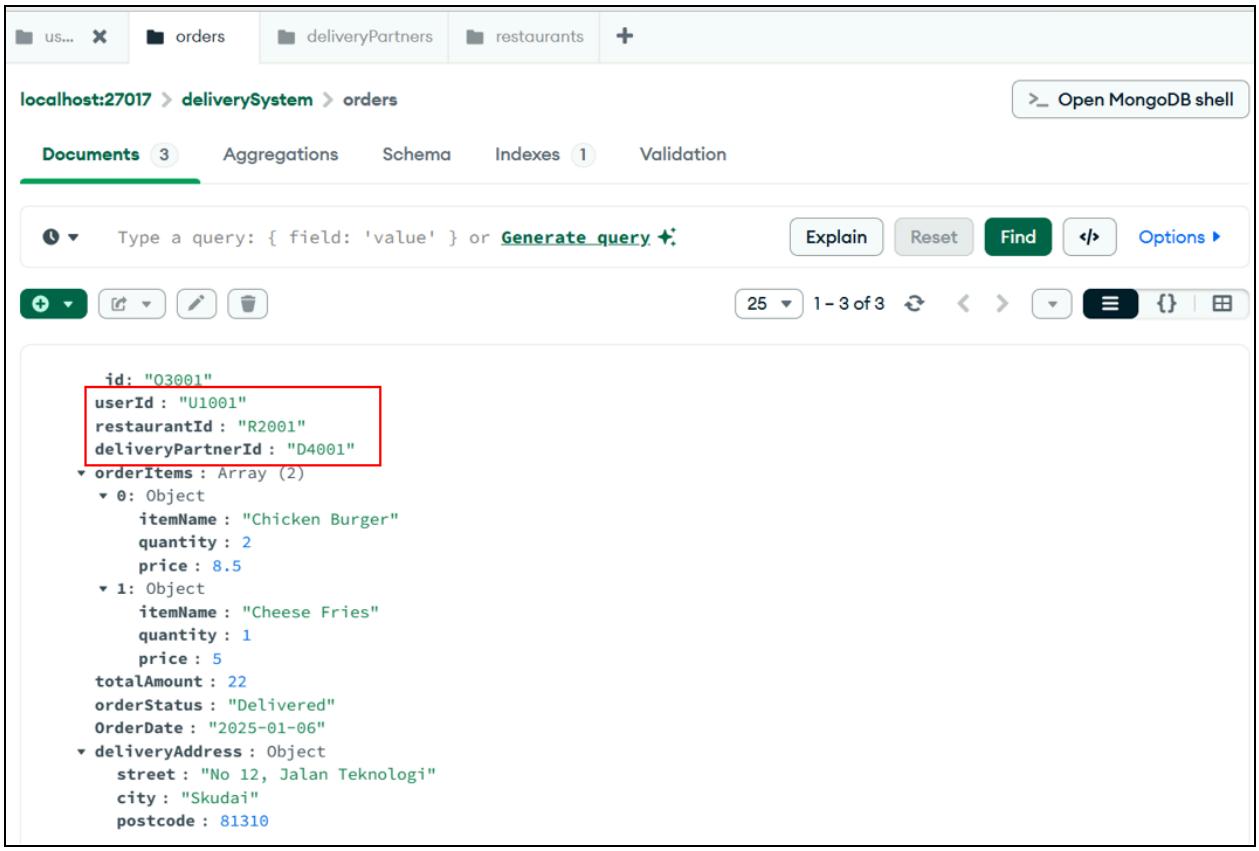


Figure 2.3.2 Reference documents

In summary, embedding prioritizes read performance and simplicity, while referencing focuses on data consistency, scalability, and reduced redundancy.

2.4 Data types used

Various types of data are being used for each document in the collection. For example, string, boolean, integer, double, array, date, and object. Figure 2.4 shows data types that are used in each collection.

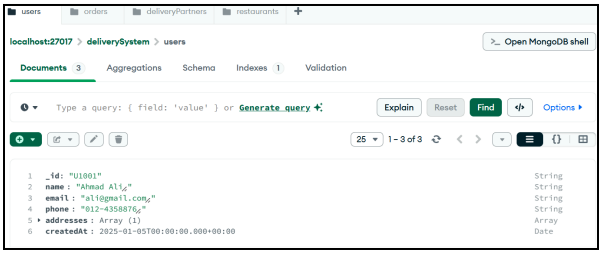


Figure 2.4.1 Data type used in users

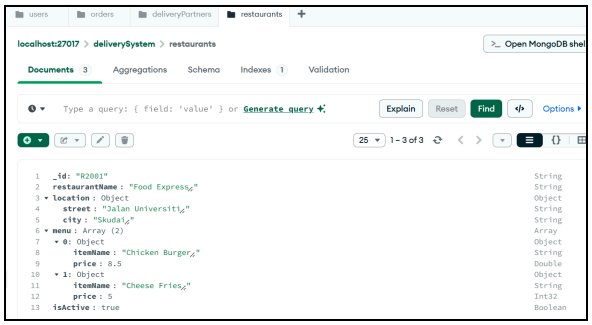


Figure 2.4.2 Data types used in restaurants

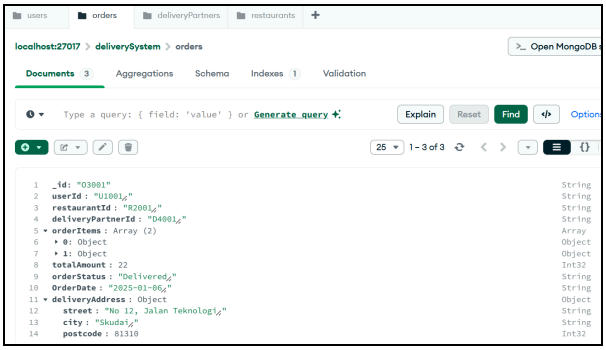
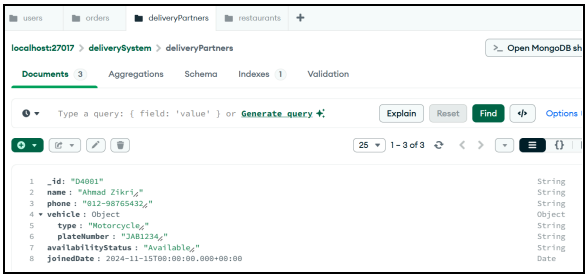


Figure 2.4.3 Data types used in orders



Data types used in deliveryPartners

3.0 CRUD Implementation Summary

This section shows the core functional capabilities of our Food Delivery System using MongoDB Compass. CRUD is the basic operations of the system where the function is created, read, updated, and deleted. Various operations have been implemented to manage the users, restaurants, orders, and delivery partners.

3.1 Explanation of insert, query, update, delete operations

The following MongoDB operations were utilized to support the system's requirements:

- **Insert Operations:** This is the process of adding new information to the database for their system. The insertOne() operation for single entries such as registering a new customer account.
- **Query Operations:** This is the process of retrieving and viewing data based on specific criteria or needs. The find() operation to search through the collections like users, restaurants, orders or deliveryPartners. Condition filters like $gt and $or were implemented to find specific data such as using $gt (Greater Than) to identify the orders with a totalAmount higher than 15.00 and used $or to find orders that are either "Preparing" or "Out for delivery". Since our data model used arrays to store menu items and order details, array queries were utilized to find documents where an array contains a specific value. For example, searching the restaurants collection for a specific itemName inside the menu array.
- **Update Operations:** This is the process of modifying existing data when circumstances change in the real world. The updateOne() and updateMany() operations are the primary methods used to target one or more documents for modification based on the specific filter criteria. It also has many update operators like $set, $inc, $push and $pull. The $set is utilized to replace the field value with a new one. For example, this operator was used to update a driver's availabilityStatus in the deliveryPartners collection from "Available" to "Busy". The $inc operator is to perform the mathematical changes by incrementing or decrementing a numeric value. This is to adjust the food prices in the restaurants collection such as increasing a menu item's price for inflation.
- **Delete Operations:** This is the process of removing data that is no longer needed or relevant to maintain the system performance. The deleteOne() operation that targets a specific record for removal like a user cancelling a single order.

3.2 Screenshots of MongoDB shell execution

This section presents the visual representation of the CRUD operations performed within the MongoDB environment. Each screenshot demonstrates the input command and the corresponding JSON response from the server.

Insert Operations

The registration of new entities is verified through the insertOne() command as shown in Figure 3.2.1, the execution of db.users.insertOne(). This figure shows the successful addition of a new user document. The shell returns an acknowledged: true response along with the insertId: 'U1004', confirming the record is now showing in the database.

Figure 3.2.1 updateOne()

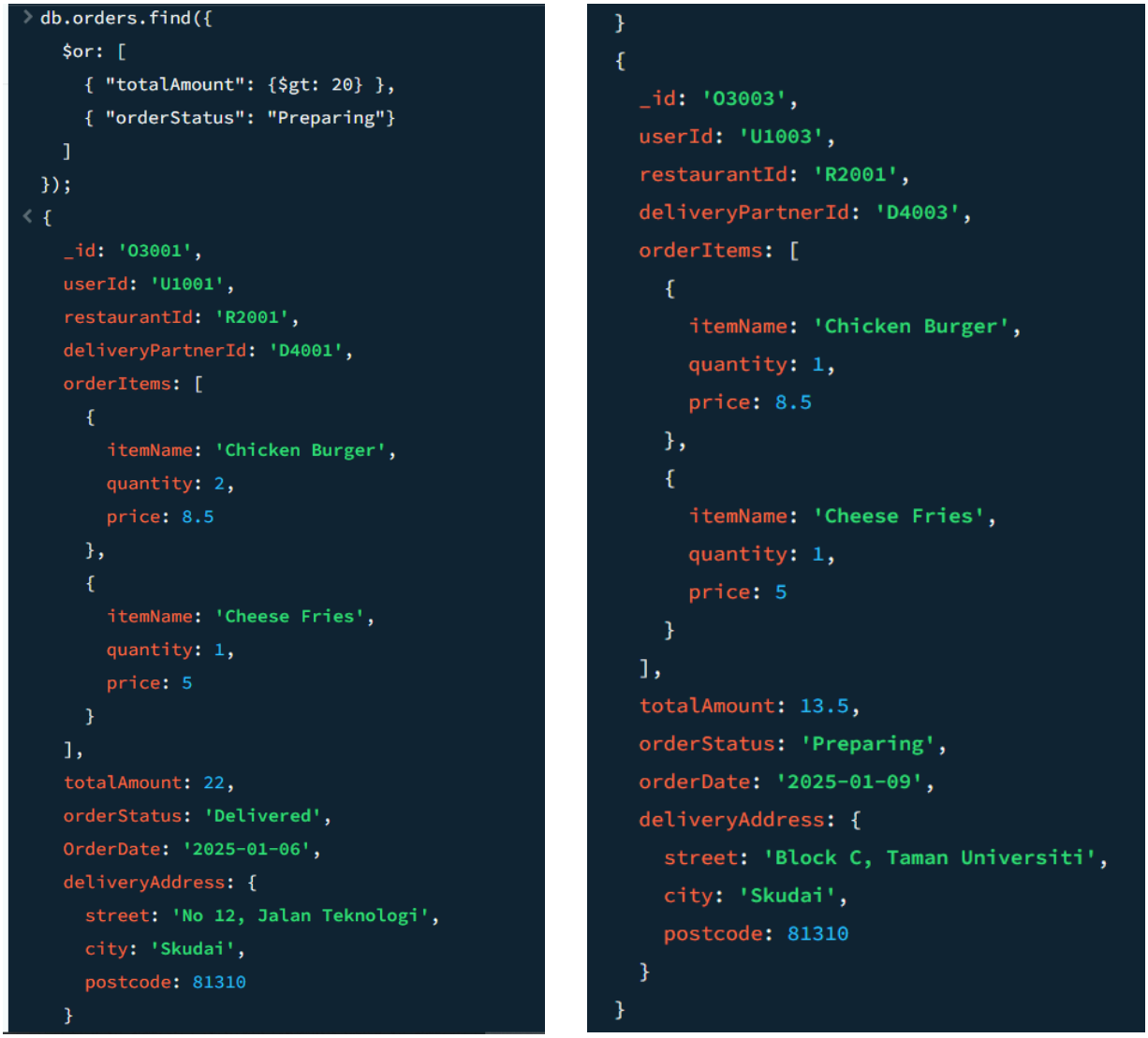
Query Operations

The ability to retrieve specific datasets is demonstrated using the operators. Figure 3.2.2 showing the query high-value orders using the find(). This query retrieves all the restaurants with an active status. By applying a projection, the output only displays the restaurantName and location, while the _id field is excluded to provide a cleaner data view.

```
> db.restaurants.find({"isActive": true}, {"restaurantName": 1,
                      "location": 1, "_id": 0});
< {
    restaurantName: 'Food Express',
    location: {
      street: 'Jalan Universiti',
      city: 'Skudai'
    }
  }
  {
    restaurantName: 'Ar-rayyan',
    location: {
      street: 'Jalan Kebudayaan',
      city: 'Johor Bahru'
    }
  }
```

Figure 3.2.2 find()

Figure 3.2.3 is using the multi-condition filtering using $or operator. The shell output displays the orders that match multiple statuses like “Preparing” or the totalAmount is greater than 20, proving that the system’s ability to handle the complex search logic.



```
> db.orders.find({
    $or: [
      { "totalAmount": {$gt: 20} },
      { "orderStatus": "Preparing"}
    ]
  });
< {
    _id: 'O3001',
    userId: 'U1001',
    restaurantId: 'R2001',
    deliveryPartnerId: 'D4001',
    orderItems: [
      {
        itemName: 'Chicken Burger',
        quantity: 2,
        price: 8.5
      },
      {
        itemName: 'Cheese Fries',
        quantity: 1,
        price: 5
      }
    ],
    totalAmount: 22,
    orderStatus: 'Delivered',
    OrderDate: '2025-01-06',
    deliveryAddress: {
      street: 'No 12, Jalan Teknologi',
      city: 'Skudai',
      postcode: 81310
    }
  }
  {
    _id: 'O3003',
    userId: 'U1003',
    restaurantId: 'R2001',
    deliveryPartnerId: 'D4003',
    orderItems: [
      {
        itemName: 'Chicken Burger',
        quantity: 1,
        price: 8.5
      },
      {
        itemName: 'Cheese Fries',
        quantity: 1,
        price: 5
      }
    ],
    totalAmount: 13.5,
    orderStatus: 'Preparing',
    orderDate: '2025-01-09',
    deliveryAddress: {
      street: 'Block C, Taman Universiti',
      city: 'Skudai',
      postcode: 81310
    }
  }
```

Figure 3.2.3 $gt and $or operator

Figure 3.2.4 validates an Array Query. The system successfully identifies “Food Express” by searching deep within the menu array for a specific itemName which is ‘Cheese Fries’. Then, it displays the restaurants that have the menu name even though it is in array terms. This demonstrates the flexibility of the Document Data Model in retrieving documents based on the nested array values.

```
> db.restaurants.find({ "menu.itemName": "Cheese Fries"});
< {
    _id: 'R2001',
    restaurantName: 'Food Express',
    location: {
      street: 'Jalan Universiti',
      city: 'Skudai'
    },
    menu: [
      {
        itemName: 'Chicken Burger',
        price: 8.5
      },
      {
        itemName: 'Cheese Fries',
        price: 5
      }
    ],
    isActive: true
  }
```

Figure 3.2.4 array queries

Update Operations

Existing data was modified using targeted operators to reflect the real-time changes in the delivery ecosystem. Figure 3.2.5 using the updateOne() and the $set operator to modify the availabilityStatus of a delivery partner. The modifiedCount: 1 confirms that the document was successfully updated from ‘Available’ to ‘Busy’ status.

The operation demonstrated in Figure 3.2.5 used the $inc operator to increase the price of all items in a restaurant’s menu by 1.00. This highlights the efficiency of using updateMany() to update multiple embedded documents at once.

Figure 3.2.5 updateOne()

Figure 3.2.6 updateMany()

Delete Operations

This is the process of removing data that is no longer needed or relevant to maintain the system performance. The deleteOne() operations targets a single and specific record for removal based on the unique identifier. Figure 3.2.7 demonstrates the removal of a specific order from the orders collection using its unique ID. While the server returns acknowledged: true, the deletedCount: 0 in this specific execution shows that the record with that ID either did not exist or had already been removed. In a successful way, the deletedCount would update to 1 as shown in figure 3.2.8.

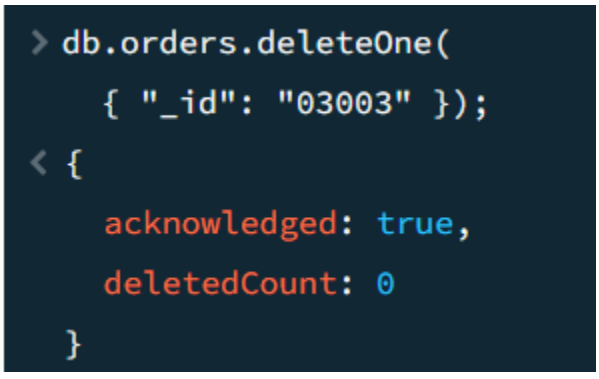


Figure 3.2.7 deleteOne() fail

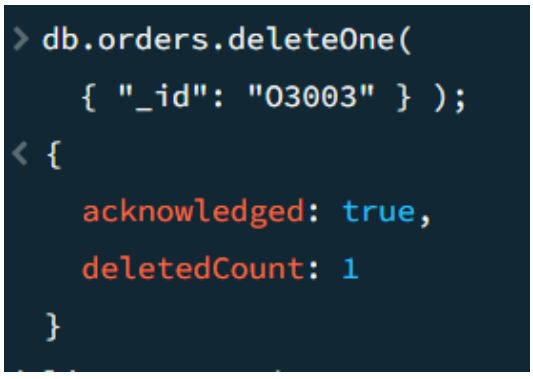


Figure 3.2.8 deleteOne() success

3.3 Projections and query tests

To improve system performance and data privacy, projections were implemented to return only the essential information. The orders collection was sorted to identify all the transactions where the totalAmount is greater than 15, as demonstrated in Figure 3.2.9. This query specifically included the userId and totalAmount fields while excluding the _id to provide a cleaner output for admin view. Additionally, a query was performed on the restaurants collection to identify which vendors offer "Chicken Burger", as illustrated in Figure 3.2.10. These operations ensure that the backend effectively handles data retrieval requirements.

```
> use deliverySystem
< switched to db deliverySystem
> db.orders.find(
    { "totalAmount": { $gt: 15 } },
    { "userId": 1, "totalAmount": 1, "_id": 0 }
  );
< {
    userId: 'U1001',
    totalAmount: 22
  }
```

Figure 3.2.9 Test 1

```
> db.restaurants.find({ "menu.itemName": "Chicken Burger" });
< {
    _id: 'R2001',
    restaurantName: 'Food Express',
    location: {
      street: 'Jalan Universiti',
      city: 'Skudai'
    },
    menu: [
      {
        itemName: 'Chicken Burger',
        price: 9.5
      },
      {
        itemName: 'Cheese Fries',
        price: 6
      }
    ],
    isActive: true
  }
```

Figure 3.2.10 Test 2

The use of the Document Data Model allowed us to nest addresses or menu items as arrays, which can reduce the need for complex join. Using $inc also ensures the updates for pricing, and prevents any errors during simultaneous transactions, while projections ensure that the sensitive user data is not unnecessarily exposed during the standard queries.

4.0 Advanced Operations

Advanced operations that have been implemented are indexing, aggregation and sorting based on the collection that has been created. Improvements and challenges for these operations have also been stated.

4.1 Index creation examples

Figure 4.1.1 shows the creation of a single-field index on the orderStatus field in the orders collection. This index allows queries filtering orders by status (e.g., “Delivered” or “Out for delivery”) to execute faster by avoiding a full collection scan.

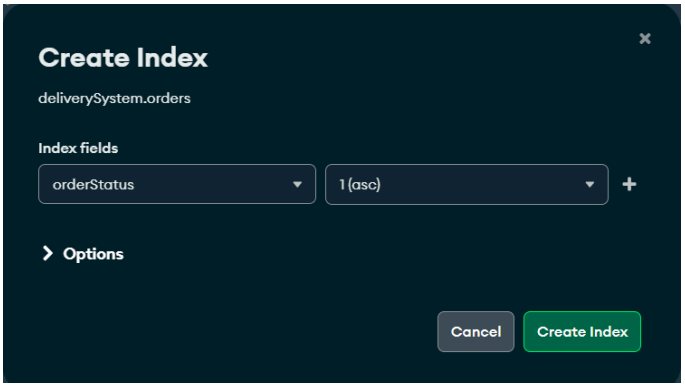


Figure 4.1.1 Single-field index created on the orderStatus field in the orders collection.

This figure demonstrates the creation of a compound index on userId and totalAmount. The compound index enables efficient retrieval of all orders for a specific user and allows sorting by total amount without scanning the entire collection.

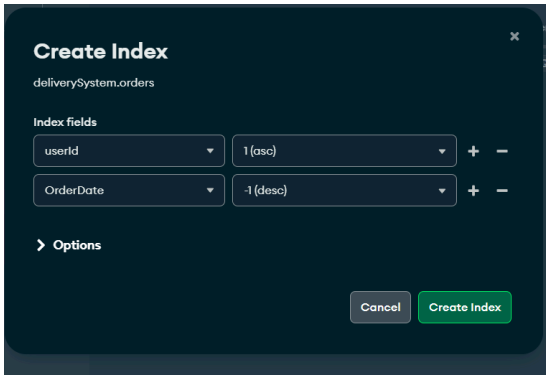


Figure 4.1.2 Compound index created on the userId and totalAmount fields in the orders collection.

Figure 4.1.3 demonstrates the creation of a compound index on userId and totalAmount. The compound index enables efficient retrieval of all orders for a specific user and allows sorting by total amount without scanning the entire collection.

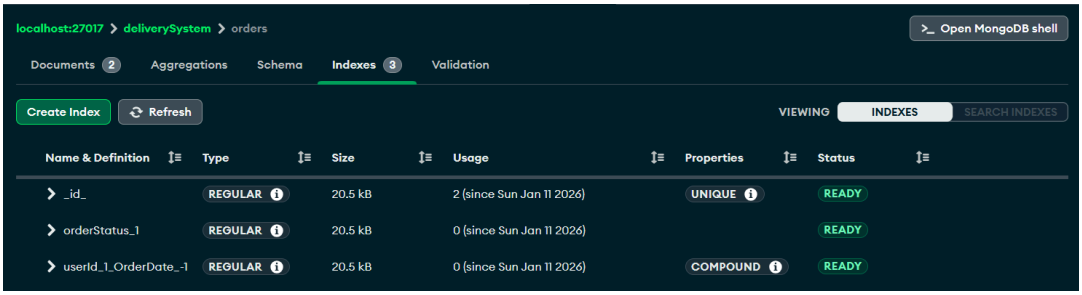


Figure 4.1.3 Index list displayed in MongoDB Compass showing the _id_, single-field, and compound indexes.

4.2 Aggregation pipeline examples

This figure shows an aggregation pipeline grouping orders by orderStatus and counting the total number of orders in each category. It demonstrates how MongoDB pipelines can summarize data and provide quick insights into order distribution.

```
> db.orders.aggregate([{$group: {_id:"$orderStatus", totalOrders: {$sum: 1}}}])
< {
    _id: 'Delivered',
    totalOrders: 1
  }
  {
    _id: 'Out for delivery',
    totalOrders: 1
  }
```

Figure 4.2.1 Aggregation pipeline to calculate total number of orders grouped by order status.

This figure shows the output of the previous aggregation pipeline with restaurants sorted in descending order of total sales. This helps identify top-performing restaurants and demonstrates the combination of $group and $sort stages.

```
> db.orders.aggregate([{$group: {_id: "$restaurantId", totalSales: {$sum: "$totalAmount"}}}, {$sort: { totalSales: -1}}])
< {
    _id: 'R2001',
    totalSales: 22
  }
  {
    _id: 'R2002',
    totalSales: 9
  }
```

Figure 4.2.2 Aggregation result showing total sales per restaurant sorted in descending order.

4.3 Sorting demonstrations

This figure shows the orders collection sorted in ascending order of totalAmount, displaying the smallest orders first. It demonstrates how sorting can help identify low-value transactions or prioritize smaller orders for review.

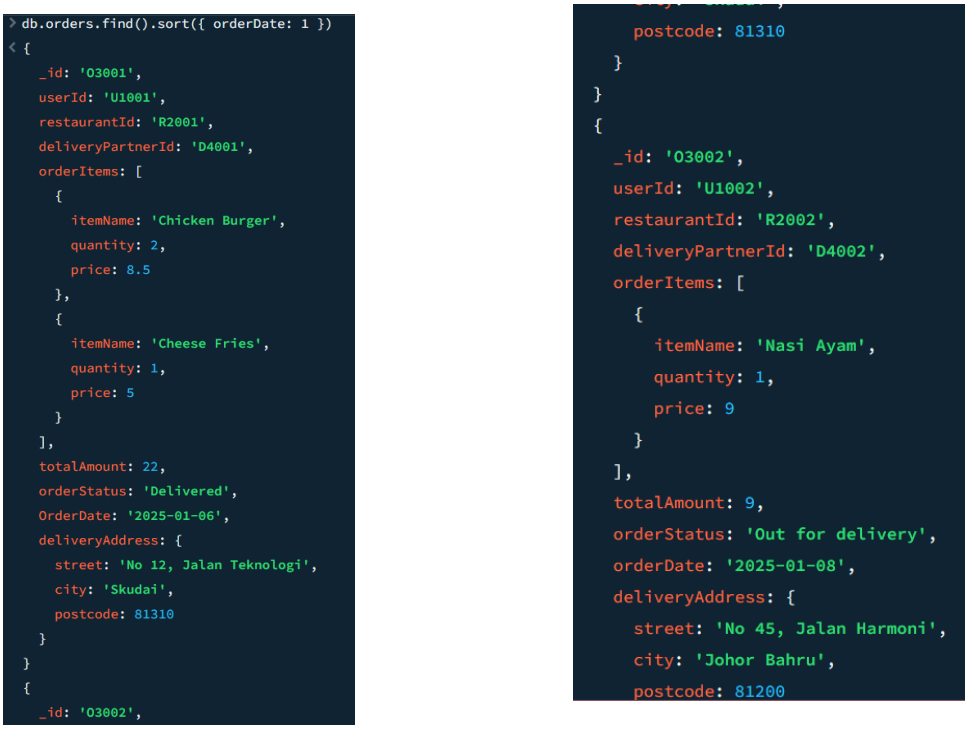


Figure 4.3.1 Sorting orders by totalAmount in ascending order

This figure shows the orders collection sorted in descending order of totalAmount, displaying the largest orders first. It highlights how sorting helps quickly identify high-value transactions for analysis or reporting.

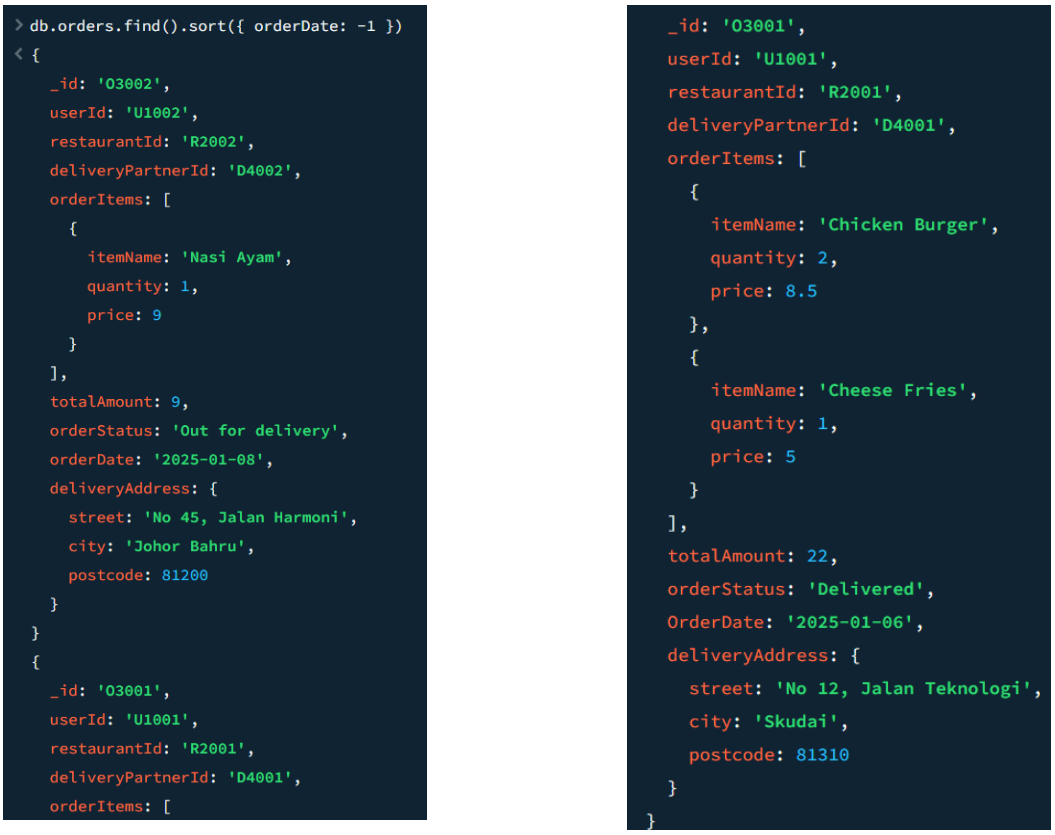


Figure 4.3.2 Sorting orders by totalAmount in descending order

4.4 Explanation of performance improvements

Indexes elevate query performance by allowing mongodb to locate documents without scanning the entire collection. The single-field index on orderStatus speeds up queries filtering by status and the compound index on usedId and orderDate allows queries that filter by a user and sort by date to use the index efficiently. Without indexes, mongodb would perform a full collection scan which is slower and uses more resources. With indexes, queries only traverse relevant parts of the btree structure.

4.5 Any challenges encountered

We encountered several difficulties when indexing, aggregation and sorting were implemented in MongoDB. First, in order to guarantee proper sorting and precise query results, certain numerical fields like totalAmount needed to be handled carefully. Second, creating compound indexes in MongoDB Compass GUI was initially confusing, as multiple fields must be added manually rather than typed together but it was sorted out pretty quickly. Lastly, selecting which fields to index and use in aggregation pipelines required careful thought to guarantee that queries were relevant to the delivery system domain and that the result was clear and concise.

5.0 Security & Limitations

5.1 Basic access controls

Basic access control is one of the most important security mechanisms in any database-driven system, especially one that manages sensitive information such as user details, orders, and delivery addresses. In MongoDB, this is commonly handled through RBAC, where users are assigned specific roles that define what actions they are allowed to perform. For example, an administrator role may have full permissions to create collections, manage indexes, and perform updates or deletions, while regular application users are restricted to only reading or updating their own data. This separation of privileges helps minimize security risks by ensuring that users cannot access or modify data beyond their responsibilities. Although this project focuses more on demonstrating database concepts rather than full system deployment, applying access control is crucial in real-world applications to prevent unauthorized access, accidental data changes, and potential misuse of system resources.

5.2 Injection risk

Injection attacks is a significant security concern in NoSQL databases, including MongoDB, particularly when user inputs are directly used in queries without proper validation. NoSQL injection occurs when attackers manipulate input fields to include MongoDB operators such as $gt, $or, or $ne, allowing them to alter query logic and potentially bypass authentication or retrieve sensitive data. In a delivery system, this could lead to unauthorized access to user accounts or order information. To reduce this risk, it is essential to validate and sanitize all user inputs at the application level before executing database queries. Using parameterized queries, enforcing strict input formats, and avoiding the direct execution of user-provided query structures can significantly improve security. While the queries in this project were executed directly through the MongoDB shell for demonstration purposes, a production-level system would require additional safeguards to protect against injection-based attacks.

5.3 Database constraints or limitations

Despite its flexibility and scalability, MongoDB also has several limitations that need to be considered. One key limitation is its schema-less nature, which allows documents within the same collection to have different structures. While this flexibility speeds up development, it can also lead to inconsistent or incomplete data if schema validation is not enforced. This may cause issues during querying, aggregation, or sorting operations when expected fields are missing or stored in incorrect data types. Additionally, handling transactions that span multiple documents or collections is more complex in MongoDB compared to traditional relational databases. Although MongoDB supports multi-document transactions, they may introduce performance overhead, especially in high-traffic systems such as delivery platforms. Another limitation is indexing, where creating too many indexes can negatively impact write performance and increase storage usage. Therefore, developers must carefully balance flexibility, performance, and data consistency when designing a NoSQL database system.

6.0 Teamwork

The following table outlines the team members' roles and contributions to this project.

Name	Contributions
Yasmin Batrisyia	By completing my task, I get to enhance my knowledge of implementing NoSQL databases using MongoDB. I had the opportunity to learn the differences between NoSQL and SQL databases and apply them in this project. For example, I learned how to create multiple collections and documents, identify data types, and differentiate between reference documents and embedded documents. Additionally, I also improved my understanding by assisting and monitoring my friends' tasks to ensure that we all had the same level of knowledge.
Syarifah Dania	I contributed to the project by implementing indexing, aggregation pipelines, and sorting demos for the delivery system database. In order to enhance query performance, I developed compound and single-field indexes and showed how they maximize searches and data retrieval. In order to summarize order data, I also created aggregation pipelines that calculate total orders by status and total revenues per restaurant. I also carried out sorting examples to demonstrate

	ascending and descending order on pertinent variables. My knowledge of MongoDB operations, query optimization, and how pipelines and indexes boost NoSQL database performance has all improved as a result of this training.
Nur Firzana	My contribution focused on developing a complete CRUD lifecycle starting with create/insert operations, read/query operations, update operations and delete operations. I learned that this is the basic operations to create the backend system. I learned about many of the operators that can make it easy for the user to easily find, search and ensure all the data is neat and organized. This experience taught me how the document data model allows for a more flexible and organized method to handle real-world data, ensuring the backend is both efficient and scalable.
Ain Nurnabila	For my part and contribution, I explained how indexing influences query performance in the delivery system, discussing collection scans vs. indexed queries and performance trade-offs. Analyzed NoSQL security concerns including access control (RBAC), injection risks with prevention methods, and consistency trade-offs in distributed systems. Discussed additional challenges such as schema flexibility, transaction limitations, and scalability considerations. Performed final report integration, language review, and formatting consistency check.

7.0 Conclusion

This project successfully demonstrated the design and implementation of a robust NoSQL backend for a modern and technology Food Delivery System. By supporting MongoDB's document-oriented architecture, the system was able to manage the complex relationships between users, restaurants, orders and delivery partners with scalability.

7.1 What was learned

Through the process of development of this NoSQL database backend, an understanding of Document Data Model was achieved. The project proved that the MongoDB is highly scalable for handling flexible data structures and real-time updates required by the modern delivery platforms.

Key technical takeaways from this project include the ability of the CRUD lifecycle, which ensures the system remains dynamic through efficient data insertion and removal. Experience was gained in enhancing the operator efficiency and effectively reducing backend complexity. Furthermore, the transition from Collection Scans (COLLSCAN) to Index Scans (IXSCAN) demonstrated how the indexing significantly improved the performance by reducing the latency and resource consumption. The implementation of Projections proved essential to maintain the data privacy and system speed by limiting query outputs to only necessary information.

7.2 Challenges

A number of challenges were encountered during the delivery system's MongoDB deployment. Making sure the right data types were used for sorting and aggregation, for example, managing numeric values like totalAmount to prevent inaccurate query results was one of the challenges. Creating compound indexes in MongoDB Compass was another challenge because it required careful attention to ensure proper indexing because various fields had to be manually added. It was also difficult to decide which fields to index and use in aggregation pipelines because the team had to make sure that queries remained efficient, relevant, and useful without overcrowding the database with pointless indexes. It also took great thought to comprehend and reduce security threats in a NoSQL context, such as operator insertion and role-based access control. Lastly, a difficulty to teamwork and organization was coordinating work among team members to combine CRUD operations, indexing, aggregation, and sorting consistently. This was resolved by task allocation and clear communication.

7.3 Recommendations for improvement

To enhance the performance, the following improvements are recommended. Firstly is the implementation of schema validation. While MongoDB offers flexibility, a production-level system should utilize JSON Schema Validation. This would enforce data integrity at the database level, ensuring that essential fields like order status attach to specific formats and required data types,to prevent the polluted data issues encountered during development.

Next is to do automated backup and continuous monitoring. Future iterations should integrate tools like MongoDB Atlas Monitoring. Establishing automated, point-in-time recovery (PITR) backups would ensure data persistence and provide deeper insights into query execution plans, helping to identify slow-running queries before they impact the user experience.

Lastly, enhanced the security protocols. Other than the basic Role-Based Access Control (RBAC), implementing Field-Level Encryption (FLE) for sensitive user data like phone numbers and addresses would ensure that even if the database is compromised, the most sensitive information remains unreadable.